LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 3 KONSEP DASAR



NAMA : NANDA MAYLA ADIASTI

NIM : 103132400003

KELAS : S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrograman Berorientasi Objek (Object Oriented Programming/OOP) merupakan paradigma pemrograman yang mengelompokkan data dan fungsi ke dalam objek. Konsep utama OOP meliputi abstraksi, enkapsulasi, inheritance, polymorphism, dan komunikasi antarobjek**.**

Abstraksi digunakan untuk mengambil karakteristik dan perilaku penting suatu objek. **E**nkapsulasi merupakan proses menggabungkan data dengan method sekaligus melindungi data dari akses yang tidak sesuai**.** Inheritance memungkinkan kelas baru mewarisi atribut dan method dari kelas induknya, sedangkan polymorphism memungkinkan satu interface digunakan untuk menghasilkan tindakan yang berbeda.

Dalam OOP terdapat class sebagai cetak biru untuk membuat objek. Class memiliki atribut untuk menyimpan data dan method untuk mendefinisikan perilaku atau proses. Sementara itu, object merupakan hasil instansiasi dari suatu class.

Method digunakan untuk membagi tugas dalam program dan dapat mengembalikan nilai maupun tidak. Selain itu, Java memiliki keyword **this** yang digunakan untuk merujuk pada objek yang sedang digunakan, terutama untuk membedakan atribut dengan parameter yang memiliki nama sama.

B. Guided

1. ProjectModul3.java

```
package com.mayla.projectmodul3;

public class ProjectModul3 {

public static void main(String[] args) {
System.out.println("Hello World!");
}
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul3 ---
Hello World!
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.275 s
Finished at: 2026-10-04T12:03:10+07:00
------------------------------------------------------------------------
```

2. Kucing.java

```
package com.mayla.projectmodul3.guided.abstraksi;

public class Kucing {
String nama;
String ras;

void bersuara(){
System.out.println(nama + ras + "meow");
 }
}
```

3. Abstraksi.java

```
package com.mayla.projectmodul3.guided.abstraksi;


public class Abstraksi {
public static void main(String[] args){
Kucing aren = new Kucing();

aren.nama = "Aren Gentong";
aren.ras = "Mujaer";
aren.bersuara();
 }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul3 ---
Aren GentongMujaermeow
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.147 s
Finished at: 2026-09-29T13:15:01+07:00
------------------------------------------------------------------------
```

4. Rekening.java

```
package com.mayla.projectmodul3.guided.enkapsulasi;
public class Rekening {
private int saldo = 0;

public void tambahSaldo(int jumlah){
saldo = saldo + jumlah;
System.out.println("Saldo berhasil ditambahkan");
}

public void tampilkanSaldo(){
System.out.println("Saldo Anda:" + saldo);
}
}
```

5. Enkapsulasi.java

```
package com.mayla.projectmodul3.guided.enkapsulasi;

public class Enkapsulasi {
public static void main(String[] args){
Rekening rek = new Rekening();

rek.tambahSaldo(20000000);
rek.tampilkanSaldo();

}
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul3 ---
Saldo berhasil ditambahkan
Saldo Anda:20000000
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.462 s
Finished at: 2026-09-29T13:34:25+07:00
------------------------------------------------------------------------
```

C. Unguided

1. Soal 1 Unguided

```
# Soal Unguided — Modul 3: Pemrograman Berorientasi Objek
### Penerapan OOP Sederhana untuk Pengolahan Data

---

## Petunjuk Umum

Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan kode.

- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya.
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga `this` perlu dipakai secara eksplisit).
- Program ini harus memuat dua jenis field: **instance field** (milik masing-masing object) dan **class field** (milik class, dipakai bersama).
- Tidak perlu inheritance, overriding, maupun abstract class.

---

## Soal — Pembersihan dan Analisis Data Suhu Mingguan

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

### Kebutuhan program

Class ini harus dapat:

1. **Menyimpan data suhu 7 hari**, dibuat dari array suhu mentah yang sudah ada di `main` (index array mewakili hari).
2. **Menyimpan nilai penanda data kosong (`-1.0`) sebagai aturan tetap milik class**, yaitu class field yang nilainya tidak dapat diubah dan dipakai bersama oleh semua object.
3. **Menampilkan seluruh data apa adanya**, menandai hari yang kosong secara jelas.
```

4. **Mencari apakah ada hari yang kosong, dan di index berapa**, karena index inilah yang nanti dipakai untuk proses pengisian data.
5. **Mengisi hari yang kosong**, menggunakan index dari poin 4, dengan nilai perkiraan berdasarkan rata-rata suhu satu hari sebelum dan satu hari sesudahnya.
6. **Menghitung rata-rata suhu** dari data yang sudah bersih.

Hint

> Hint di bawah ini adalah acuan minimal. Anda tetap boleh menambah method atau menyesuaikan detail selama seluruh kebutuhan di atas terpenuhi.

Nama class: `PengolahSuhu`

Field yang disarankan:

Field	Jenis	Tipe	Keterangan
`suhuHarian`	Instance field	`double[]`	Menyimpan array suhu 7 hari; berbeda untuk setiap object
`NILAI_KOSONG`	Class field	`double`	Nilai penanda data kosong (`-1.0`); sama untuk semua object dan tidak berubah

Method yang disarankan:

Method	Kebutuhan	Return
`tampilkanData()`	3	`void`
`cariIndexKosong()`	4	`int` — index hari yang bernilai `NILAI_KOSONG`, atau `-1` apabila tidak ditemukan
`isiDataKosong()`	5	`void` — memanggil `cariIndexKosong()` di dalamnya; jika hasilnya bukan `-1`, isi index tersebut
`hitungRataRata()`	6	`double`

Rumus imputasi (pengisian data kosong) untuk time series:

Untuk index hari `i` yang kosong:

```
suhuHarian[i] = ( suhuHarian[i-1] + suhuHarian[i+1] ) / 2
```

Ini adalah metode imputasi paling dasar untuk data deret waktu, yaitu **mengisi nilai yang hilang dengan rata-rata dari titik data tetangga terdekatnya** (sebelum dan sesudah), dengan asumsi nilai yang hilang berada di antara dua kondisi yang sudah diketahui dan tidak berubah drastis.

Data suhu (array di `main`)

Gunakan array berikut sebagai data suhu 7 hari (Hari 1 = index 0):

```java
double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };
```

Hari yang kosong: **Hari 4**.

Alur program di `main`

1. Siapkan array `suhuHarian` seperti di atas.
2. Buat object `PengolahSuhu` dari array `suhuHarian` tersebut. Tampilkan data awal (`tampilkanData()`), lalu panggil `cariIndexKosong()` dan tampilkan index yang ditemukan.
3. Jalankan `isiDataKosong()`.
4. Tampilkan data akhir yang sudah bersih (`tampilkanData()`) beserta rata-rata suhunya.
5. Tampilkan kembali isi array `suhuHarian` di `main` (bukan lewat object `PengolahSuhu`, melainkan variabel array yang sudah disiapkan di langkah 1). Amati bahwa array ini pun sudah berubah, padahal tidak pernah diubah langsung di `main`, lalu jelaskan sebabnya dalam beberapa kalimat sebagai komentar pada kode program.

Output yang diharapkan

```
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

        Index hari kosong (dimulai dari 0): 3

        === Data Suhu Setelah Pengisian ===
        Hari 1 : 30.4°C
        Hari 2 : 24.3°C
        Hari 3 : 26.8°C
        Hari 4 : 29.1°C
        Hari 5 : 31.4°C
        Hari 6 : 30.8°C
        Hari 7 : 32.9°C

        Rata-rata : 29.39°C
```

```
112.
113.           Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
114.           [30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
115.           (ikut berubah: constructor menyimpan referensi array yang sama)
116.           ```
117.
118.           ---
119.
120.           ## Yang perlu diperhatikan saat mengerjakan
121.
122.           - Amati baik-baik dua baris terakhir pada output di atas. Array `suhuHarian` yang dicetak itu
                adalah variabel di `main`, bukan diambil lewat object `PengolahSuhu` — dan nilainya sudah berubah,
                padahal `main` tidak pernah memanggil operasi pengubahan itu sendiri. Ini terjadi karena array
                bertipe **reference**: constructor `PengolahSuhu` menyimpan array yang sama persis (bukan
                menyalinnya), sehingga perubahan yang dilakukan `isiDataKosong()` pada object otomatis terlihat
                juga dari sisi `main`. Jelaskan hal ini dalam beberapa kalimat sebagai komentar pada kode program.
123.           - Nilai penanda `-1.0` harus ditulis satu kali saja sebagai class field, lalu dipakai oleh setiap
                method yang membutuhkannya. Jangan mengetik ulang angka `-1.0` di dalam method.
124.           - Rumus imputasi mengasumsikan hari sebelum dan sesudah hari yang kosong tersedia
                sebagai data asli (bukan ikut kosong). Data pada soal ini sudah dijamin memenuhi asumsi tersebut.
125.
```

PengolahSuhu.java

```
package unguided;

public class PengolahSuhu {
    // Instance field
   // Setiap object PengolahSuhu memiliki array suhu masing-masing.
   private double[] suhuHarian;

   // Class field
   // Nilainya sama untuk semua object dan tidak dapat diubah.
   private static final double NILAI_KOSONG = -1.0;

   // Constructor
   // Nama parameter sama dengan nama field, sehingga menggunakan this.
   public PengolahSuhu(double[] suhuHarian) {
      this.suhuHarian = suhuHarian;
   }

   // Menampilkan seluruh data suhu.
   public void tampilkanData() {
      for (int i = 0; i < suhuHarian.length; i++) {

         System.out.print("Hari " + (i + 1) + " : ");

         if (suhuHarian[i] == NILAI_KOSONG) {
            System.out.println("(kosong)");
         } else {
            System.out.printf("%.1f°C%n", suhuHarian[i]);
         }
      }
   }

   // Mencari index yang berisi nilai kosong.
   public int cariIndexKosong() {
      for (int i = 0; i < suhuHarian.length; i++) {

         if (suhuHarian[i] == NILAI_KOSONG) {
            return i;
         }
      }

      // Jika tidak ada data kosong.
      return -1;
   }

   // Mengisi data yang kosong menggunakan rata-rata
   // suhu satu hari sebelum dan satu hari sesudahnya.
   public void isiDataKosong() {

      int indexKosong = cariIndexKosong();
```

```
                if (indexKosong != -1) {

                    suhuHarian[indexKosong] =
                            (suhuHarian[indexKosong - 1]
                            + suhuHarian[indexKosong + 1]) / 2;
                }
            }

            // Menghitung rata-rata suhu setelah data bersih.
            public double hitungRataRata() {

                double total = 0;

                for (double suhu : suhuHarian) {
                    total += suhu;
                }

                return total / suhuHarian.length;
            }
}
```

Main.java

```
package unguided;

import java.util.Arrays;

public class Main {

    public static void main(String[] args) {

        // 1. Menyiapkan array suhu 7 hari.
        double[] suhuHarian = {
            30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9
        };

        // 2. Membuat object PengolahSuhu.
        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        // Menampilkan data awal.
        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();

        // Mencari index data yang kosong.
        int indexKosong = pengolah.cariIndexKosong();

        System.out.println();
        System.out.println(
            "Index hari kosong (dimulai dari 0): " + indexKosong
        );

        // 3. Mengisi data yang kosong.
        pengolah.isiDataKosong();

        // 4. Menampilkan data setelah pengisian.
        System.out.println();
        System.out.println("=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();

        // Menampilkan rata-rata suhu.
        System.out.printf(
            "%nRata-rata : %.2f°C%n",
            pengolah.hitungRataRata()
        );

        // 5. Menampilkan kembali array suhuHarian yang ada di main.
        System.out.println();
        System.out.println(
            "Isi array suhuHarian di main setelah "
            + "isiDataKosong() dijalankan:"
        );
```

```
        System.out.println(Arrays.toString(suhuHarian));

        /*
         * Penjelasan:
         * Array suhuHarian yang ada di main ikut berubah karena
         * constructor PengolahSuhu menyimpan reference/referensi
         * ke array yang sama, bukan membuat salinan array baru.
         *
         * Ketika method isiDataKosong() mengubah isi array melalui
         * object pengolah, perubahan tersebut sebenarnya dilakukan
         * pada array yang sama dengan array suhuHarian di main.
         *
         * Oleh karena itu, setelah isiDataKosong() dijalankan,
         * nilai pada array suhuHarian di main juga ikut berubah.
         */
        System.out.println(
            "(ikut berubah: constructor menyimpan referensi array yang sama)"
        );
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul3 ---
=== Data Suhu Awal ===
Hari 1 : 30,4°C
Hari 2 : 24,3°C
Hari 3 : 26,8°C
Hari 4 : (kosong)
Hari 5 : 31,4°C
Hari 6 : 30,8°C
Hari 7 : 32,9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30,4°C
Hari 2 : 24,3°C
Hari 3 : 26,8°C
Hari 4 : 29,1°C
Hari 5 : 31,4°C
Hari 6 : 30,8°C
Hari 7 : 32,9°C

Rata-rata : 29,39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.285 s
Finished at: 2026-10-04T11:45:12+07:00
------------------------------------------------------------------------
```

Penjelasan :

Program ini menggunakan konsep Pemrograman Berorientasi Objek (OOP) dengan membuat class PengolahSuhu untuk mengolah data suhu selama 7 hari. Class tersebut memiliki suhuHarian sebagai instance field yang menyimpan data suhu masing-masing object, serta NILAI_KOSONG sebagai class field static final yang digunakan sebagai penanda data kosong dan nilainya tidak dapat diubah.

Constructor PengolahSuhu digunakan untuk menerima array suhu dari main. Pada constructor digunakan this.suhuHarian = suhuHarian karena nama parameter sama dengan nama field. Method tampilkanData() digunakan untuk menampilkan data suhu dan memberikan keterangan (kosong) apabila ditemukan nilai penanda. Method cariIndexKosong() mencari posisi data yang kosong dan mengembalikan index-nya. Selanjutnya, method isiDataKosong() mengisi data tersebut menggunakan rata-rata suhu pada hari sebelum dan sesudahnya. Method hitungRataRata() digunakan untuk menghitung rata-rata seluruh suhu setelah data kosong berhasil diisi.

Pada program ini, array dari main tidak disalin ketika diberikan ke object PengolahSuhu, melainkan referensinya digunakan secara langsung. Oleh karena itu, ketika isiDataKosong() mengubah nilai pada array melalui object, array suhuHarian yang berada di main juga ikut berubah. Hal tersebut terjadi karena keduanya menunjuk pada array yang sama di memori.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep dasar Pemrograman Berorientasi Objek (OOP) dapat diterapkan dalam bahasa Java melalui class, object, field, constructor, dan method. Program yang dibuat berhasil mengolah data suhu selama 7 hari dengan mencari data kosong, mengisinya menggunakan rata-rata suhu sebelum dan sesudahnya, serta menghitung rata-rata suhu setelah data lengkap. Praktikum ini juga menunjukkan penggunaan instance field, class field, dan keyword this, serta memahami bahwa array yang diberikan kepada object menggunakan referensi yang sama sehingga perubahan pada object juga memengaruhi array di main.